

Today

- Virtual memory
 - Questions 1-4
- Multi-processors
 - Questions 5-8

(Time allowing)

- Scheduling
 - Questions 9-10, 12-13
- I/O
 - Questions 14-15

Virtual memory

1. What effect does increasing the page size have?

- greater internal fragmentation (space lost within pages)
- + TLB will cover more data (fixed no. of entries but more data due to increased page size)
- + greater I/O throughput (more data per fault/sequential access)
- pages may contain more updated data
- page fault latency increases (more I/O per fault)
- working set size may increase
- + decreases the number of pages: less pages, less page table entries (and potentially faster page table lookup)

2. Why is demand paging generally more prevalent than pre-paging?

- Demand paging: load data on vm-fault
- Pre-paging: load data before it's requested
- (It's difficult to predict the future! Predicting incorrectly wastes compute resources.)

3. Describe four replacement policies and compare them.

Optimal

- Remove pages that won't be used in the near future
- Impossible to always accurately predict the future, so not feasible to implement

LRU (least recently used)

- More difficult to implement
- Requires maintaining timestamps, but feasible to implement

FIFO (first in, first out)

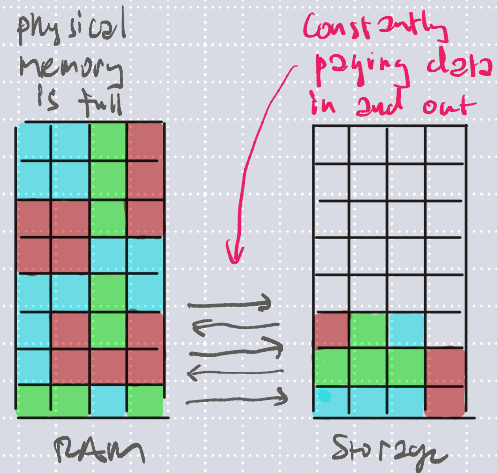
- Simple to implement
- May replace pages that are about to be used

Clock

- need to maintain a reference bit
- less chance of removing an actively used page

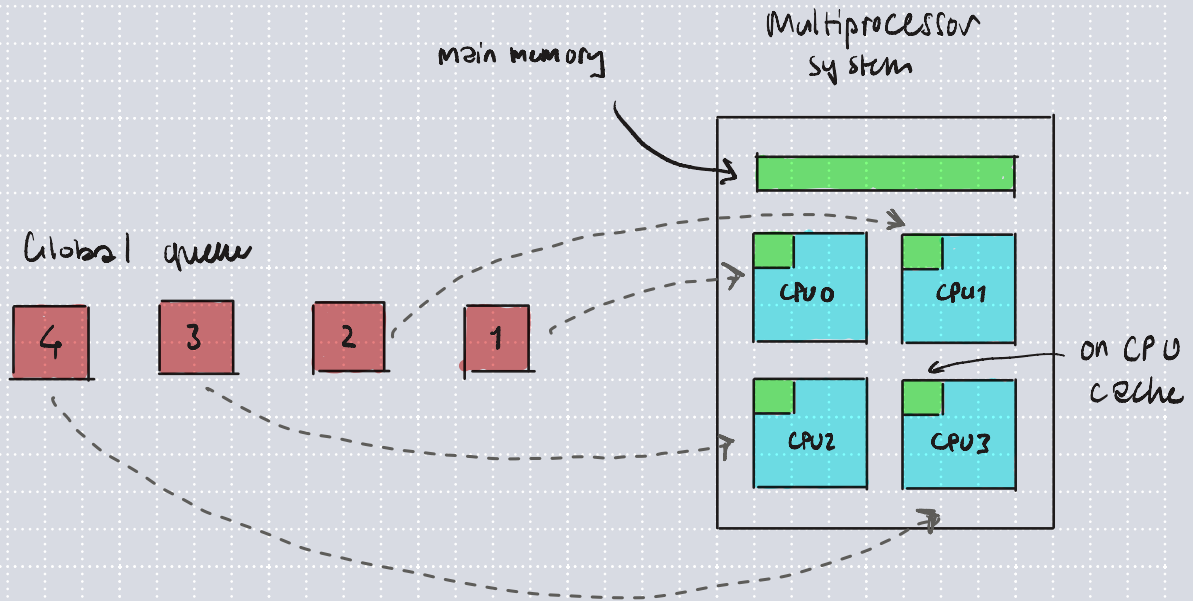
4. What is thrashing? How can it be detected? What can be done to combat it?

- Thrashing occurs when the working set of the currently active processes exceeds physical memory; as a consequence, much time is spent paging data in and out.
- It can be detected by monitoring the page fault rate.
- It can be combatted by swapping entire processes to disk or (more drastically) by killing processes.



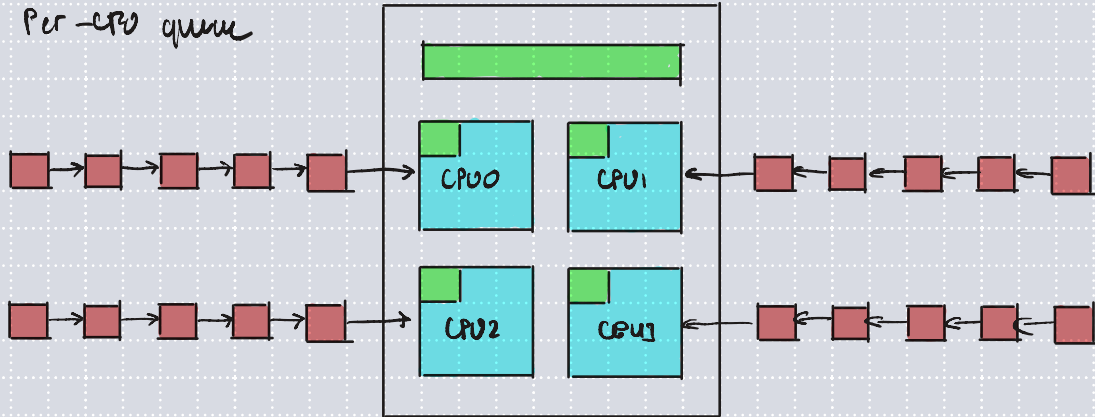
Multi-processors

5. What are the advantages and disadvantages of using a global scheduling queue over per-CPU queues? Under which circumstances would you use the one or the other? What features of a system would influence this decision?



5. What are the advantages and disadvantages of using a global scheduling queue over per-CPU queues? Under which circumstances would you use the one or the other? What features of a system would influence this decision?

Per-CPU queue



Global

- + Automatic load balancing
- Contention on queue itself
- Doesn't utilise hot caches (that is data may be cached on one CPU but not another)

Per-CPU

- + CPU affinity (better cache usage)
- + reduces queue contention
- load balancing is more difficult

6. When does spinning on a lock (busy waiting, as opposed to blocking on the lock, and being woken up when it's free) make sense in a multiprocessor environment?

- If it would take less time to spin and acquire the lock then it would be better to block and then be woken up, it makes sense to spin
- Work while a spinlock is held is generally fast

7. Why is preemption an issue with spinlocks?

- If a process takes the lock and is preempted, it can't release the lock until it is next scheduled
- Solution: disable interrupts while the lock is held

8. How does a read-before-test-and-set lock work and why does it improve scalability?

- Also known as test-and-test-and-set lock (TTSL)
- Spin on local cached copy of lock
- Prevents bus contention caused by repeated attempts at TSL instruction
- Cache invalidation will cause the local copies to get updated once lock is released

Scheduling

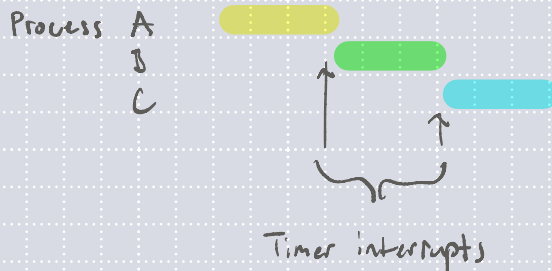
9. What do the terms I/O bound and CPU bound mean when used to describe a process (or thread)?

CPU bound: more CPU time would significantly increase progress

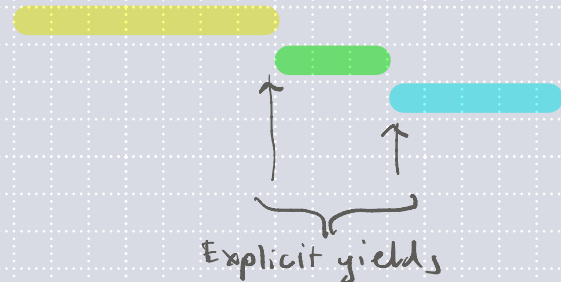
I/O bound: more CPU time would not significantly increase progress; progress is dependent on I/O progress

10. What is the difference between cooperative and pre-emptive multitasking?

Preemptive case



Cooperative case



12. Why would a hypothetical OS always schedule a thread in the same address space over a thread in a different address space? Is this a good idea?

- The context switch doesn't involve swapping address spaces, so it's less expensive
- This could result in starvation for other processes

13. Why would a round robin scheduler NOT use a very short time slice to provide good responsive application behaviour?

- As time slice duration decreases, a greater proportion of the time is spent context switching between processes (context switch isn't free)

I/O

14. Describe programmed I/O and interrupt-driven I/O in the case of receiving input (e.g. from a serial port). Which technique normally leads to more efficient use of the CPU? Describe a scenario where the alternative technique is more efficient.

- programmed I/O involves (typically) polling a status register waiting for completion
- interrupt-driven I/O avoids polling and instead, the CPU is interrupted upon I/O completion
- interrupt-driven I/O is typically more performant, but if I/O happens quickly (e.g. network interface cards) then programmed I/O can be performant too

15. A device driver routine (e.g. `read_block()` from disk) is invoked by the file system code. The data for the filesystem is requested from the disk, but is not yet available. What do device drivers generally do in this scenario?

- Block on a synchronisation primitive.
- An interrupt will occur later once the data is available, which will unblock the driver.